

OBJEKTINIS PROGRAMAVIMAS

Generated by Doxygen 1.13.2

1 ObjektinisProgramavimas	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Class Documentation	9
5.1 Studentas Class Reference	9
5.1.1 Member Function Documentation	10
5.1.1.1 print()	10
5.1.1.2 readFromConsole()	10
5.2 StudentasMasyvas Struct Reference	10
5.3 Zmogus Class Reference	10
6 File Documentation	13
6.1 Includes.h	13
6.2 Struktura.h	13
Index	17

Chapter 1

Objektinis Programavimas

v1.5 README:

Klases [Studentas](#) testas:

Abstrakcios klases zmogus testas:

v1.2 README:

[Studentas](#) klasę galima rasti "Struktura.h" faile

Įvestis/Išvestis iš [Studentas](#) klasės: readFromConsole metodas (Įvesties metodas iš konsolės):

Pirmiausia, naudojamas `std::cin` norint nuskaityti studento vardą ir pavardę. Po to, vartotojas prašomas įvesti studento pažymius, kol nebus įvestas -1 (tai bus ženklas, kad įvedimas baigiasi). Galiausiai, prašoma įvesti egzamino pažymį, kuris bus paskutinis įvestas duomenų taškas.

readFromFile metodas (Įvesties metodas iš failo):

Šis metodas skaito informaciją iš failo. Failo struktūra turi būti tokia, kad kiekvienas studento įrašas būtų eilutėje su vardu, pavarde ir pažymiais.

Veikimas:

Pirmiausia, metodas nuskaitys studento vardą ir pavardę. Tada jis nuskaitys visus pažymius ir įrašys juos į pažymiai vektorių. Paskutinis nuskaitytas pažymys bus laikomas egzamino pažymiu, ir jis bus išimtas iš pažymių sąrašo (naudojant `pop_back`).

Išvedimo operatorius `<<`: Šis operatorius (operator `<<`) naudojamas objektų [Studentas](#) duomenų išvedimui tiek į konsolę, tiek į failą. Šis operatorius priima `std::ostream` tipo srautą (tai gali būti tiek `std::cout` (konsolė), tiek `std::ofstream` (failas)) ir formatuoja išvedimą pagal nurodytus reikalavimus.

1.1 README:

Class vs Struct

Kompiliavimo flag'ų palyginimas (greitis)

Kompiliavimo flagų palyginias (dydis)

1.0 README:

Pagrindiniai programos naudojimo aspektai:

Ši programa leidžia efektyviai atlikti tam tikrus veiksmus su studentų sąrašu (juos išrūšiuoti, išskirstyti į skirtingus failus, jiems generuoti vardus, pažymius ir pan.). Taip pat šia programa galima stebėti skirtingų konteinerių, tokių kaip vector, deque ar list veikimo efektyvumą, taip pat ir skirtingų algoritmų pritaikymą. Programa tu MakeLists.txt failą, kuris leidžia lengvai sugeneruoti naudojamą failą.

Programos veikimo laikas, lyginant vector, list ir deque:

Testavimo sistemos parametrai: CPU - i5-1335u (10 cores, 12 threads, 12M Cache, up to 4.60 GHz)

RAM - DDR4 8GB, 2667 MHz

DISK - NVMe SSD, read speed up to 3500 MB/s, write speed up to 1625 MB/s

detalūs lyginimo duomenys:

Su vektoriais:

Su Listais:

Su Deque:

STRATEGIJŲ PALYGINIMAS:

Pirma strategija - Vektoriaus išskaidymas į du atskirus vektorius

Antra strategija - Dalis vektoriaus perkeliama į kitą, tada ištrinama iš pirmos (Dažni trynimai buvo itin skausmingi programos efektyvumui, todėl ji buvo lėčiausia)

Trečia strategija - Optimizuota pirmą strategiją

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

StudentasMasyvas	10
Zmogus	10
Studentas	9

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	9
StudentasMasyvas	10
Zmogus	10

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

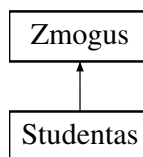
Includes.h	13
Struktura.h	13

Chapter 5

Class Documentation

5.1 Studentas Class Reference

Inheritance diagram for Studentas:



Public Member Functions

- **Studentas** (const [Studentas](#) &other)
- [Studentas](#) & **operator=** (const [Studentas](#) &other)
- **Studentas** ([Studentas](#) &&other) noexcept
- [Studentas](#) & **operator=** ([Studentas](#) &&other) noexcept
- string **getVardas** () const
- string **getPavarde** () const
- vector< int > & **getPazymiai** ()
- double **getEgzaminoPazimys** () const
- double **getGalutinisBalasVidurkis** () const
- double **getGalutinisBalasMediana** () const
- void **setVardas** (const string &v)
- void **setPavarde** (const string &p)
- void **setEgzaminoPazimys** (double egzaminoPazimys)
- void **setGalutinisBalasVidurkis** (double balas)
- void **setGalutinisBalasMediana** (double balas)
- void [readFromConsole](#) () override
- void **readFromFile** (istream &in)
- void [print](#) () const override

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) (const [Zmogus](#) &other)
- [Zmogus](#) & **operator=** (const [Zmogus](#) &other)
- **Zmogus** ([Zmogus](#) &&other) noexcept
- [Zmogus](#) & **operator=** ([Zmogus](#) &&other) noexcept

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string **vardas**
- string **pavarde**

5.1.1 Member Function Documentation

5.1.1.1 `print()`

```
void Studentas::print () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

5.1.1.2 `readFromConsole()`

```
void Studentas::readFromConsole () [inline], [override], [virtual]
```

Implements [Zmogus](#).

The documentation for this class was generated from the following file:

- Struktura.h

5.2 StudentasMasyvas Struct Reference

Public Attributes

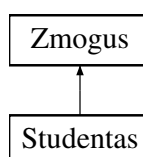
- string **vardas**
- string **pavarde**
- int * **pazymiai**
- int **pazymiuSkaicius**
- int **pazymiuCapacity**
- int **egzaminoPazimys**

The documentation for this struct was generated from the following file:

- DinaminiaiMasyvai.cpp

5.3 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- **Zmogus** (const [Zmogus](#) &other)
- [Zmogus](#) & **operator=** (const [Zmogus](#) &other)
- **Zmogus** ([Zmogus](#) &&other) noexcept
- [Zmogus](#) & **operator=** ([Zmogus](#) &&other) noexcept
- virtual void **readFromConsole** ()=0
- virtual void **print** () const =0

Protected Attributes

- string **vardas**
- string **pavarde**

The documentation for this class was generated from the following file:

- Struktura.h

Chapter 6

File Documentation

6.1 Includes.h

```
00001
00002 #ifndef INCLUDES_H
00003 #define INCLUDES_H
00004
00005 #include <iostream>
00006 #include <vector>
00007 #include <iomanip>
00008 #include <algorithm>
00009 #include <random>
00010 #include <fstream>
00011 #include <sstream>
00012 #include <chrono>
00013 #include <filesystem>
00014 #include <cstdlib>
00015 #include <ctime>
00016 #include <sstream>
00017 #include "Struktura.h"
00018 #include <list>
00019 #include <deque>
00020
00021 using std::cout;
00022 using std::string;
00023 using std::cin;
00024 using std::endl;
00025 using std::vector;
00026 using std::left;
00027 using std::setw;
00028 using std::setprecision;
00029 using std::fixed;
00030
00031
00032
00033 int generateRandomNumber(int min, int max);
00034 string generateRandomName();
00035 string generateRandomSurname();
00036 void ApskaiciuotiVeikimoLaikoVidurki();
00037 template <typename Container>
00038 void sortStudentai(Container& studentai, int RusiavimoPasirinkimas, int DuomenuKiekis = 0);
00039 void generateFile(int numStudents);
00040 void runDinaminiaiMasyvai();
00041 void WithRandomNumbers();
00042 void GenerateRandomNumbersAndNames();
00043 void WithVectors();
00044 void WithRandomNumbersArrays(int);
00045 void SuListais(int);
00046 void sortStudentaiList(std::list<Studentas>& studentai, int RusiavimoPasirinkimas);
00047 void SuDeque(int);
00048
00049
00050 #endif
00051
```

6.2 Struktura.h

```
00001 #include <iostream>
```

```

00002 #include <vector>
00003 #include <string>
00004 #include <iomanip>
00005
00006 using namespace std;
00007
00008 #ifndef ZMOGUS_H
00009 #define ZMOGUS_H
00010
00011 class Zmogus {
00012 public:
00013     // Default constructor
00014     Zmogus() = default;
00015
00016     // Virtual destructor
00017     virtual ~Zmogus() = default;
00018
00019     // Copy constructor
00020     Zmogus(const Zmogus& other)
00021         : vardas(other.vardas), pavarde(other.pavarde) {}
00022
00023     // Copy assignment operator
00024     Zmogus& operator=(const Zmogus& other) {
00025         if (this != &other) {
00026             vardas = other.vardas;
00027             pavarde = other.pavarde;
00028         }
00029         return *this;
00030     }
00031
00032     // Move constructor
00033     Zmogus(Zmogus&& other) noexcept
00034         : vardas(std::move(other.vardas)), pavarde(std::move(other.pavarde)) {}
00035
00036     // Move assignment operator
00037     Zmogus& operator=(Zmogus&& other) noexcept {
00038         if (this != &other) {
00039             vardas = std::move(other.vardas);
00040             pavarde = std::move(other.pavarde);
00041         }
00042         return *this;
00043     }
00044
00045     virtual void readFromConsole() = 0;
00046     virtual void print() const = 0;
00047
00048 protected:
00049     string vardas;
00050     string pavarde;
00051 };
00052
00053 #endif // ZMOGUS_H
00054
00055 #ifndef STRUKTURA_H
00056 #define STRUKTURA_H
00057
00058 class Studentas : public Zmogus {
00059 private:
00060     vector<int> pazymiai;
00061     double egzaminoPazimys = 0;
00062     double galutinisBalasVidurkis = 0;
00063     double galutinisBalasMediana = 0;
00064
00065 public:
00066     // Default constructor
00067     Studentas() = default;
00068
00069     // Destructor
00070     ~Studentas() = default;
00071
00072     // Copy constructor
00073     Studentas(const Studentas& other)
00074         : Zmogus(other),
00075           pazymiai(other.pazymiai),
00076           egzaminoPazimys(other.egzaminoPazimys),
00077           galutinisBalasVidurkis(other.galutinisBalasVidurkis),
00078           galutinisBalasMediana(other.galutinisBalasMediana) {}
00079
00080     // Copy assignment operator
00081     Studentas& operator=(const Studentas& other) {
00082         if (this != &other) {
00083             Zmogus::operator=(other);
00084             pazymiai = other.pazymiai;
00085             egzaminoPazimys = other.egzaminoPazimys;
00086             galutinisBalasVidurkis = other.galutinisBalasVidurkis;
00087         }
00088     }

```

```

00089         galutinisBalasMediana = other.galutinisBalasMediana;
00090     }
00091     return *this;
00092 }
00093
00094 // Move constructor
00095 Studentas(Studentas&& other) noexcept
00096 : Zmogus(std::move(other)),
00097   pazymiai(std::move(other.pazymiai)),
00098   egzaminoPazimys(other egzaminoPazimys),
00099   galutinisBalasVidurkis(other.galutinisBalasVidurkis),
00100   galutinisBalasMediana(other.galutinisBalasMediana) {}
00101
00102 // Move assignment operator
00103 Studentas& operator=(Studentas&& other) noexcept {
00104     if (this != &other) {
00105         Zmogus::operator=(std::move(other));
00106         pazymiai = std::move(other.pazymiai);
00107         egzaminoPazimys = other egzaminoPazimys;
00108         galutinisBalasVidurkis = other.galutinisBalasVidurkis;
00109         galutinisBalasMediana = other.galutinisBalasMediana;
00110     }
00111     return *this;
00112 }
00113
00114 // Getters
00115 string getVardas() const { return vardas; }
00116 string getPavarde() const { return pavarde; }
00117 vector<int>& getPazymiai() { return pazymiai; }
00118 double getEgzaminoPazimys() const { return egzaminoPazimys; }
00119 double getGalutinisBalasVidurkis() const { return galutinisBalasVidurkis; }
00120 double getGalutinisBalasMediana() const { return galutinisBalasMediana; }
00121
00122 // Setters
00123 void setVardas(const string& v) { vardas = v; }
00124 void setPavarde(const string& p) { pavarde = p; }
00125 void setEgzaminoPazimys(double egzaminoPazimys) { this->egzaminoPazimys = egzaminoPazimys; }
00126 void setGalutinisBalasVidurkis(double balas) { galutinisBalasVidurkis = balas; }
00127 void setGalutinisBalasMediana(double balas) { galutinisBalasMediana = balas; }
00128
00129 //read from console
00130 void readFromConsole() override {
00131     pazymiai.clear();
00132     cout << "Vardas: ";
00133     cin >> vardas;
00134     cout << "Pavarde: ";
00135     cin >> pavarde;
00136
00137     cout << "Iveskite pazymius (iveskite -1 kad baigti):\n";
00138     int paz;
00139     while (cin >> paz && paz != -1) {
00140         pazymiai.push_back(paz);
00141     }
00142     cout << "Iveskite egzamino pazymi:\n";
00143     cin >> egzaminoPazimys;
00144 }
00145
00146 // read from file
00147 void readFromFile(istream& in) {
00148     pazymiai.clear();
00149     in >> vardas >> pavarde;
00150
00151     int paz;
00152     while (in >> paz) {
00153         pazymiai.push_back(paz);
00154     }
00155
00156     egzaminoPazimys = pazymiai.back();
00157     pazymiai.pop_back();
00158 };
00159
00160 //print
00161 void print() const override {
00162     cout << left << setw(20) << vardas
00163           << setw(20) << pavarde
00164           << setw(15) << galutinisBalasVidurkis
00165           << setw(15) << galutinisBalasMediana
00166           << endl;
00167 }
00168 };
00169
00170 #endif

```


Index

ObjektinisProgramavimas, [1](#)

print
 Studentas, [10](#)

readFromConsole
 Studentas, [10](#)

Studentas, [9](#)
 print, [10](#)
 readFromConsole, [10](#)
StudentasMasyvas, [10](#)

Zmogus, [10](#)